DBMS Complete Study Notes

Database Management Systems — Interview + Exam Preparation

Theory + SQL Syntax + Real-World Design

PHASE 1: DBMS FUNDAMENTALS (~20%)

Goal: Database ka andar kya hota hai — yeh samajhna. Yahan hum core building blocks cover karenge jo sab kuch ka base hai.

1.1 DBMS vs RDBMS

DBMS (Database Management System) ek software system hai jo data ko store, retrieve, aur manage karne ka kaam karta hai. RDBMS (Relational DBMS) DBMS ka ek specific type hai jisme data tables (rows & columns) ke form mein store hota hai aur tables aapas mein relationships se jude hote hain.

DBMS	RDBMS
Data ko tables mein store KARNA ZARURI nahi	Data hamesha tables mein store hota hai
Tables ke beech relationships ho bhi sakti hain, nahi bhi	Tables Foreign Keys se linked hoti hain
SQL support optional ho sakti hai	SQL (Structured Query Language) use hoti hai
Example: File System, XML DB	Example: MySQL, PostgreSQL, Oracle, SQL Server
Normalization ka concept nahi hota	Normalization follow ki jaati hai

> 📝 *Note: Interview mein agar poochha jaaye — RDBMS, DBMS ka hi ek type hai. Aap hamesha yeh clarify karo.*

1.2 Data, Schema aur Instance

Yeh teen concepts bahut basic lagte hain par confuse ho jaate hain. Clear karte hain:

- **Data: Actual values jo database mein store hoti hain. Example: 'Rahul', 101, 'Delhi'**
  - Yeh woh information hai jo hum actually store karte hain
  - Data time ke saath change hota rehta hai
- **Schema: Database ki structure/blueprint. Yeh define karta hai kaunse tables honge, kaunse columns honge, data types kya honge.**
  - Schema rarely change hota hai — ek baar design ho jaane ke baad
  - Example: Student(id INT, name VARCHAR(50), age INT)
- **Instance: Kisi particular time par database mein jo data hota hai. Ek specific snapshot.**
  - Instance time ke saath change hota hai kyunki data insert/delete/update hota hai

> 📝 *Note: Schema = Building ka blueprint. Instance = Building mein abhi jo log rehte hain. Data = Har ek cheez jo andar hai.*

1.3 Tables, Rows aur Columns

RDBMS mein sab kuch tables ke form mein hota hai — ek 2D structure jaise Excel spreadsheet.

Term	Explanation
Table (Relation)	Data ka ek logical group — ek entity ko represent karta hai. Example: Students table
Row (Tuple)	Ek single record/entry. Example: (1, 'Rahul', 20, 'Delhi') — ek student ki info
Column (Attribute)	Ek specific property ya field. Example: name, age, city
Degree	Table mein columns ki total count
Cardinality	Table mein rows ki total count

1.4 Primary Key aur Foreign Key

Primary Key

Primary Key ek ya ek se zyada columns ka combination hai jo har row ko uniquely identify karta hai.

- Hamesha UNIQUE hoti hai — do rows ka same primary key nahi ho sakta
- NULL value allowed NAHI hai primary key mein
- Ek table mein sirf EK primary key hoti hai
- Example: Student table mein student_id primary key hai

```
-- Primary Key syntax:
CREATE TABLE Students (
    student_id   INT          PRIMARY KEY,
    name         VARCHAR(50)  NOT NULL,
    age          INT,
    city         VARCHAR(30)
);

-- Ya alag se define karo:
CREATE TABLE Students (
    student_id   INT,
    name         VARCHAR(50),
    CONSTRAINT pk_student PRIMARY KEY (student_id)
);
```

Foreign Key

Foreign Key ek column hai jo kisi doosri table ki Primary Key ko refer karta hai. Yeh tables ke beech relationship banata hai.

- Foreign Key ki value ya to referenced table mein exist karni chahiye ya NULL honi chahiye
- Yeh Referential Integrity maintain karta hai
- Example: Orders table mein customer_id, Customers table ki primary key hai

```
CREATE TABLE Orders (
    order_id      INT     PRIMARY KEY,
    customer_id   INT,
    amount        DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);

-- ON DELETE CASCADE: parent delete hone par child bhi delete
-- ON DELETE SET NULL: parent delete hone par child ka FK NULL ho jaata hai
FOREIGN KEY (customer_id) REFERENCES Customers(customer_id) ON DELETE CASCADE
```

1.5 Constraints

Constraints woh rules hain jo database mein data ki accuracy aur reliability ensure karte hain. Yeh data integrity ke liye use hote hain.

Constraint	Explanation
NOT NULL	Column mein NULL value allowed nahi. Example: name NOT NULL
UNIQUE	Column ki saari values different honi chahiye. NULL allowed hai. Example: email UNIQUE
PRIMARY KEY	NOT NULL + UNIQUE ka combination. Row ko uniquely identify karta hai
FOREIGN KEY	Doosri table ki PK ko reference karta hai. Referential integrity maintain karta hai
CHECK	Custom condition define karo. Example: age > 0 AND age < 120
DEFAULT	Agar value provide na ki jaaye to default value use hogi. Example: status DEFAULT 'active'

```
CREATE TABLE Employees (
    emp_id      INT             PRIMARY KEY,
    name        VARCHAR(100)    NOT NULL,
    email       VARCHAR(100)    UNIQUE,
    age         INT             CHECK (age >= 18 AND age <= 65),
    salary      DECIMAL(10,2)   CHECK (salary > 0),
```

```
    dept        VARCHAR(50)     DEFAULT 'General',
    manager_id  INT             REFERENCES Employees(emp_id)   -- self ref
);
```

1.6 Data Models

Relational Model

Relational Model sabse common aur widely-used data model hai. Ismein:

- Data tables (relations) mein store hota hai
- Har table mein rows (tuples) aur columns (attributes) hote hain
- Tables primary/foreign keys se linked hote hain
- SQL is model ke saath use hoti hai

ER Model (Entity-Relationship Model)

ER Model ek conceptual model hai jo database ka visual representation deta hai design phase mein. Actual implementation se pehle yeh design kiya jaata hai.

- Entity: Real-world object — jaise Student, Product, Order
- Attribute: Entity ki properties — jaise Student ke name, age, email
- Relationship: Entities ke beech connection — Student 'enrolls in' Course

PHASE 1 (cont.): ER Diagram — Bahut Important!

ER Diagram database design ka pehla step hai. Interviews mein frequently pucha jaata hai — 'Is system ka ER diagram banao.'

1.7 ER Diagram Components

Entities

- Strong Entity: Khud apna existence rakhti hai. Example: Student, Product
  - Rectangle se represent hoti hai
- Weak Entity: Strong entity par depend karti hai bina uske exist nahi kar sakti
  - Double rectangle se represent hoti hai
  - Example: Loan_Payment (Loan ke bina exist nahi kar sakti)

Attributes

Attribute Type	Description
Simple	Aur divide nahi ho sakti. Example: age, id
Composite	Parts mein divide hoti hai. Example: name = first + last
Derived	Doosri attribute se calculate hoti hai. Example: age from DOB (dashed oval)
Multi-valued	Multiple values ho sakti hain. Example: phone_numbers (double oval)
Key Attribute	Uniquely identify karti hai entity ko. Example: student_id (underlined)

Relationships

- 1:1 (One to One): Ek entity ek doosri entity se related hai. Example: Employee — ParkingSpot
- 1:M (One to Many): Ek entity multiple entities se related. Example: Customer — Orders
- M:M (Many to Many): Multiple entities multiple entities se related. Example: Student — Course

Cardinality aur Participation

Concept	Explanation

Cardinality	Kitni instances ek relationship mein participate kar sakti hain (1:1, 1:M, M:N)
Total Participation	Har entity instance ko relationship mein participate KARNA PADEGA. Double line se show hoti hai
Partial Participation	Participation optional hai. Single line se show hoti hai

Practice: Student Management ER Design

Real example dekhte hain — Student Management System ka ER diagram step-by-step:

- **Entities: Student, Course, Department, Instructor**
- **Attributes:**
  - Student: student_id (key), name (composite: first+last), dob, email (multi-valued possible), phone
  - Course: course_id (key), course_name, credits, description
  - Instructor: instructor_id (key), name, specialization
  - Department: dept_id (key), dept_name, location
- **Relationships:**
  - Student ENROLLS_IN Course (M:N) — ek student multiple courses, ek course multiple students
  - Instructor TEACHES Course (1:N) — ek instructor multiple courses, ek course ek instructor
  - Department OFFERS Course (1:N) — ek dept multiple courses
  - Student BELONGS_TO Department (M:1)

```
-- Student Management System SQL Tables:
CREATE TABLE Department (
    dept_id     INT         PRIMARY KEY,
    dept_name   VARCHAR(50) NOT NULL,
    location    VARCHAR(50)
);

CREATE TABLE Student (
    student_id  INT          PRIMARY KEY,
    first_name  VARCHAR(30)  NOT NULL,
    last_name   VARCHAR(30)  NOT NULL,
    dob         DATE,
    email       VARCHAR(100) UNIQUE,
    dept_id     INT          REFERENCES Department(dept_id)
);

CREATE TABLE Course (
    course_id   INT          PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    credits     INT          CHECK (credits > 0),
    dept_id     INT          REFERENCES Department(dept_id)
```

```
);

-- M:N Relationship — Junction Table
CREATE TABLE Enrollment (
    student_id  INT  REFERENCES Student(student_id),
    course_id   INT  REFERENCES Course(course_id),
    grade       CHAR(1),
    PRIMARY KEY (student_id, course_id)   -- Composite PK
);
```

PHASE 2: SQL MASTERY (~25%)

Goal: SQL queries likhna — yeh tumhara sabse powerful weapon hai. Theory samajhna aur actual syntax likhna dono important hai.

2.1 Basic SQL

SELECT, WHERE, ORDER BY

SELECT statement se hum data retrieve karte hain. Yeh SQL ka core statement hai.

```
-- Basic SELECT
SELECT * FROM Students;                          -- sab columns
SELECT name, age, city FROM Students;            -- specific columns
SELECT DISTINCT city FROM Students;              -- duplicate cities hatao

-- WHERE clause — conditions filter karo
SELECT * FROM Students WHERE age > 18;
SELECT * FROM Students WHERE city = 'Delhi';
SELECT * FROM Students WHERE age > 18 AND city = 'Delhi';
SELECT * FROM Students WHERE age < 18 OR city = 'Mumbai';
SELECT * FROM Students WHERE NOT city = 'Chennai';

-- ORDER BY — results sort karo
SELECT * FROM Students ORDER BY name ASC;        -- A to Z
SELECT * FROM Students ORDER BY age DESC;        -- bade se chhote
SELECT * FROM Students ORDER BY city ASC, name DESC;  -- multiple columns

-- LIMIT — sirf N rows chahiye
SELECT * FROM Students ORDER BY age DESC LIMIT 5;  -- top 5 oldest
```

LIKE, BETWEEN, IN

```
-- LIKE — pattern matching
SELECT * FROM Students WHERE name LIKE 'R%';      -- R se shuru
SELECT * FROM Students WHERE name LIKE '%kumar';   -- kumar se khatam
SELECT * FROM Students WHERE name LIKE '%ah%';     -- beech mein 'ah'
SELECT * FROM Students WHERE name LIKE '_ahul';    -- 1 char + 'ahul'

-- BETWEEN — range filter (inclusive dono ends)
SELECT * FROM Students WHERE age BETWEEN 18 AND 25;
SELECT * FROM Orders WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31';

-- IN — multiple values mein se koi ek
SELECT * FROM Students WHERE city IN ('Delhi', 'Mumbai', 'Bangalore');
SELECT * FROM Students WHERE city NOT IN ('Chennai', 'Kolkata');

-- IS NULL / IS NOT NULL
```

```
SELECT * FROM Students WHERE phone IS NULL;
SELECT * FROM Students WHERE email IS NOT NULL;
```

2.2 JOINs — Bahut Important!

JOINs multiple tables ko combine karne ke liye use hote hain. Interview mein yeh zaroor poochha jaata hai. Pehle example tables dekhte hain:

```
-- Customers Table:
-- cust_id | name     | city
-- 1       | Rahul    | Delhi
-- 2       | Priya    | Mumbai
-- 3       | Vikram   | Pune

-- Orders Table:
-- order_id | cust_id | amount
-- 101      | 1       | 500
-- 102      | 1       | 300
-- 103      | 2       | 800
-- 104      | 5       | 200    <-- cust_id 5 exist nahi karta
```

INNER JOIN

Sirf woh rows return karta hai jo DONO tables mein matching condition ke saath exist karti hain.

```
SELECT c.name, o.order_id, o.amount
FROM Customers c
INNER JOIN Orders o ON c.cust_id = o.cust_id;

-- Result:
-- name  | order_id | amount
-- Rahul | 101      | 500
-- Rahul | 102      | 300
-- Priya | 103      | 800
-- (Vikram aur Order 104 dono out — no match)
```

LEFT JOIN (LEFT OUTER JOIN)

LEFT table ki saari rows return karega. Right table mein match nahi hone par NULL aayega.

```
SELECT c.name, o.order_id, o.amount
FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id;

-- Result:
-- name   | order_id | amount
-- Rahul  | 101      | 500
-- Rahul  | 102      | 300
-- Priya  | 103      | 800
-- Vikram | NULL     | NULL    <-- Vikram ka koi order nahi, phir bhi aayega
```

```
-- Use case: Woh customers dhundho jinke koi orders nahi hain
SELECT c.name FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id
WHERE o.order_id IS NULL;
```

RIGHT JOIN (RIGHT OUTER JOIN)

RIGHT table ki saari rows return karega. Left table mein match nahi hone par NULL aayega. (LEFT JOIN se opposite)

```
SELECT c.name, o.order_id, o.amount
FROM Customers c
RIGHT JOIN Orders o ON c.cust_id = o.cust_id;

-- Result:
-- name  | order_id | amount
-- Rahul | 101      | 500
-- Rahul | 102      | 300
-- Priya | 103      | 800
-- NULL  | 104      | 200    <-- Order 104 aayega, customer ka NULL
```

SELF JOIN

Ek table apne aap se join karti hai. Yeh hierarchical data ke liye use hota hai jaise Employee-Manager relationship.

```
-- Employees table mein manager_id bhi employee_id hai
SELECT e.name AS Employee, m.name AS Manager
FROM Employees e
LEFT JOIN Employees m ON e.manager_id = m.emp_id;

-- Result:
-- Employee  | Manager
-- Rahul     | Priya
-- Ankit     | Priya
-- Priya     | NULL     <-- top level manager, koi manager nahi
```

2.3 Advanced SQL

GROUP BY + HAVING

GROUP BY data ko groups mein organize karta hai aur usually aggregate functions ke saath use hota hai. HAVING GROUP BY ke baad filter lagata hai (WHERE group par kaam nahi karta).

```
-- Har city mein kitne students hain?
SELECT city, COUNT(*) AS student_count
FROM Students
GROUP BY city;

-- Sirf woh cities dikhao jahan 5 se zyada students hain
SELECT city, COUNT(*) AS student_count
FROM Students
```

```
GROUP BY city
HAVING COUNT(*) > 5;

-- WHERE vs HAVING:
-- WHERE: GROUP BY se PEHLE filter lagata hai (individual rows)
-- HAVING: GROUP BY ke BAAD filter lagata hai (groups par)
SELECT city, AVG(age) AS avg_age
FROM Students
WHERE age > 15               -- pehle rows filter karo
GROUP BY city
HAVING AVG(age) > 20;        -- phir groups filter karo
```

Subqueries

Subquery ek query ke andar doosri query hoti hai. Yeh WHERE, FROM, ya SELECT clause mein ho sakti hai.

```
-- WHERE mein subquery (most common):
-- Woh students dhundho jinki age average se zyada hai
SELECT name, age FROM Students
WHERE age > (SELECT AVG(age) FROM Students);

-- IN ke saath subquery:
-- Woh students dhundho jinhone Math course liya hai
SELECT name FROM Students
WHERE student_id IN (
    SELECT student_id FROM Enrollment
    WHERE course_id = (SELECT course_id FROM Courses WHERE course_name =
'Math')
);

-- FROM mein subquery (Derived Table):
SELECT dept, avg_salary FROM (
    SELECT dept, AVG(salary) AS avg_salary
    FROM Employees
    GROUP BY dept
) AS dept_averages
WHERE avg_salary > 50000;
```

EXISTS, ANY, ALL

```
-- EXISTS: Check karo kya subquery koi result return karti hai
-- Woh customers jinhone koi order diya hai
SELECT name FROM Customers c
WHERE EXISTS (
    SELECT 1 FROM Orders o WHERE o.cust_id = c.cust_id
);

-- ANY: Subquery ki kisi bhi value se comparison
-- Woh employees jinki salary kisi bhi manager se zyada hai
SELECT name FROM Employees
WHERE salary > ANY (SELECT salary FROM Employees WHERE role = 'Manager');
```

```
-- ALL: Subquery ki saari values se comparison
-- Woh employee jinki salary sabse zyada hai
SELECT name FROM Employees
WHERE salary >= ALL (SELECT salary FROM Employees);
```

CASE Statement

CASE SQL ka if-else hai. Conditional logic add karne ke liye use hota hai.

```
-- Simple CASE:
SELECT name, age,
    CASE
        WHEN age < 18 THEN 'Minor'
        WHEN age BETWEEN 18 AND 60 THEN 'Adult'
        ELSE 'Senior'
    END AS age_group
FROM Students;

-- Grade calculate karo:
SELECT student_id,
    CASE
        WHEN marks >= 90 THEN 'A'
        WHEN marks >= 80 THEN 'B'
        WHEN marks >= 70 THEN 'C'
        WHEN marks >= 60 THEN 'D'
        ELSE 'F'
    END AS grade
FROM Results;
```

2.4 Functions

Aggregate Functions

```
-- COUNT: Rows count karo
SELECT COUNT(*) FROM Students;                    -- saari rows
SELECT COUNT(phone) FROM Students;               -- non-NULL phones
SELECT COUNT(DISTINCT city) FROM Students;       -- unique cities

-- SUM, AVG, MIN, MAX:
SELECT SUM(amount) FROM Orders;                  -- kul sales
SELECT AVG(salary) FROM Employees;               -- average salary
SELECT MIN(age), MAX(age) FROM Students;         -- youngest & oldest

-- Combination:
SELECT dept,
    COUNT(*)      AS total_employees,
    AVG(salary)   AS avg_salary,
    MAX(salary)   AS max_salary,
    MIN(salary)   AS min_salary
FROM Employees
GROUP BY dept;
```

String Functions

```
-- UPPER, LOWER, LENGTH:
SELECT UPPER(name) FROM Students;                -- 'rahul' -> 'RAHUL'
SELECT LOWER(email) FROM Students;               -- 'EMAIL@X.COM' ->
'email@x.com'
SELECT LENGTH(name) FROM Students;               -- character count

-- SUBSTRING / SUBSTR:
SELECT SUBSTRING(name, 1, 3) FROM Students;      -- pehle 3 characters

-- CONCAT: strings join karo
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM Students;

-- TRIM: spaces hatao
SELECT TRIM('  hello  ');                         -- 'hello'
SELECT LTRIM(name) FROM Students;                -- left spaces hatao

-- REPLACE:
SELECT REPLACE(phone, '-', '') FROM Students;    -- hyphens hatao
```

Date Functions

```
-- Current Date/Time:
SELECT NOW();                                     -- 2024-03-21 10:30:00
SELECT CURDATE();                                 -- 2024-03-21
SELECT CURTIME();                                 -- 10:30:00

-- YEAR, MONTH, DAY nikalo:
SELECT YEAR(dob), MONTH(dob), DAY(dob) FROM Students;

-- DATEDIFF: do dates ke beech days ka difference
SELECT DATEDIFF(NOW(), order_date) AS days_since_order FROM Orders;

-- DATE_ADD, DATE_SUB:
SELECT DATE_ADD(order_date, INTERVAL 30 DAY) AS delivery_date FROM Orders;

-- Age calculate karo:
SELECT name, TIMESTAMPDIFF(YEAR, dob, CURDATE()) AS age FROM Students;
```

2.5 Indexing Basics

Index ek data structure hai jo queries ko fast banata hai. Sochlo yeh book ka index hai — pura book padhe bina directly page par ja sakte ho.

Index Type	Description
Clustered Index	Data physically sort karke store hota hai. Table par sirf EK clustered index ho sakta hai. Usually Primary Key par automatically banta hai.

Non-Clustered Index	Separate structure hota hai jo actual data ko point karta hai. Table par multiple non-clustered indexes ho sakte hain. Manually create karte hain.

```
-- Index create karo:
CREATE INDEX idx_student_name ON Students(name);

-- Unique Index (UNIQUE constraint jaisa kaam karta hai):
CREATE UNIQUE INDEX idx_email ON Students(email);

-- Composite Index (multiple columns):
CREATE INDEX idx_name_city ON Students(name, city);

-- Index drop karo:
DROP INDEX idx_student_name ON Students;

-- Index kab use karo:
-- 1. WHERE clause mein frequently use hone wale columns
-- 2. JOIN conditions mein use hone wale columns
-- 3. ORDER BY mein use hone wale columns
```

PHASE 3: NORMALIZATION + DESIGN (~20%)

Goal: Database tables sahi se design karna. Yeh interviews mein sabse zyada pucha jaata hai — 'Yeh table normalize karo' ya 'Is design mein kya problem hai?'

3.1 Normalization kyu?

Bina normalization ke database mein yeh problems aate hain:

Problem	Explanation
Data Redundancy	Same data multiple jagah store hona. Agar ek jagah update karo to dusri jagah miss ho jaata hai
Insert Anomaly	Nayi info insert nahi kar sakte bina irrelevant data ke. Example: New course add nahi kar sakte bina kisi student ke
Update Anomaly	Ek cheez update karo to har row update karni padegi. Bhool gaye to inconsistency
Delete Anomaly	Kuch data delete karo to doosri zaruri info bhi chali jaati hai

3.2 Normal Forms

1NF — First Normal Form

Rule: Har cell mein atomic (indivisible) value honi chahiye. Koi repeating groups ya multi-valued attributes nahi.

```
-- 1NF VIOLATION (phone mein multiple values):
-- student_id | name  | phone
-- 1          | Rahul | 9876543210, 8765432109

-- 1NF CORRECT — har phone ek alag row mein:
-- student_id | name  | phone
-- 1          | Rahul | 9876543210
-- 1          | Rahul | 8765432109

-- Ya phone alag table mein rakho (better approach):
CREATE TABLE StudentPhones (
    student_id  INT  REFERENCES Students(student_id),
    phone       VARCHAR(15),
    PRIMARY KEY (student_id, phone)
);
```

2NF — Second Normal Form

Rule: Table 1NF mein honi chahiye + Koi Partial Dependency nahi hona chahiye. Partial Dependency tab hoti hai jab koi non-key attribute sirf composite primary key ke PART par dependent ho.

```
-- 2NF VIOLATION:
-- Composite PK: (student_id, course_id)
-- student_id | course_id | student_name | course_name | grade

-- Partial Dependencies:
-- student_name depends only on student_id (not full PK)
-- course_name depends only on course_id (not full PK)
-- grade depends on FULL PK (student_id, course_id) -- yeh OK hai

-- 2NF CORRECT — 3 tables banao:
-- Students(student_id PK, student_name)
-- Courses(course_id PK, course_name)
-- Enrollment(student_id FK, course_id FK, grade)  -- PK is composite
```

3NF — Third Normal Form

Rule: Table 2NF mein honi chahiye + Koi Transitive Dependency nahi honi chahiye. Transitive Dependency tab hoti hai jab non-key attribute kisi doosre non-key attribute par depend kare.

```
-- 3NF VIOLATION:
-- Employee(emp_id PK, name, dept_id, dept_name, dept_location)

-- Transitive Dependency:
-- dept_name depends on dept_id (which is a non-key attribute)
-- dept_location depends on dept_id
-- Both should be in a separate Departments table!

-- 3NF CORRECT:
-- Department(dept_id PK, dept_name, dept_location)
-- Employee(emp_id PK, name, dept_id FK)  -- sirf dept_id rakho
```

BCNF — Boyce-Codd Normal Form

Rule: 3NF se thoda stricter. Har functional dependency X -> Y ke liye X ek Super Key hona chahiye.

```
-- BCNF VIOLATION:
-- Course(student, subject, teacher)
-- Ek teacher sirf ek subject padhata hai
-- Ek student ek subject ke liye sirf ek teacher hota hai

-- FDs: (student, subject) -> teacher
```

```
--       teacher -> subject
-- Yahan 'teacher' -> subject mein teacher super key nahi hai — VIOLATION!

-- BCNF CORRECT — 2 tables:
-- TeacherSubject(teacher PK, subject)
-- StudentTeacher(student, teacher FK, PRIMARY KEY(student, teacher))
```

> *Note: Memory trick: 1NF=Atomic values. 2NF=No partial dependency. 3NF=No transitive dependency. BCNF=Every determinant is super key.*

3.3 Denormalization

Kuch situations mein hum intentionally normalization rules break karte hain performance improve karne ke liye. Iska naam Denormalization hai.

- When to use: Jab read operations bahut zyada hain aur JOINs slow ho rahe hain
- When to use: Data warehousing aur reporting databases mein
- When NOT to use: OLTP systems mein jahan write operations frequent hain

> *Note: Denormalization ek deliberate tradeoff hai — storage aur consistency ki cost par speed milti hai.*

3.4 Keys — Deep Dive

Key Type	Explanation
Super Key	Columns ka ek set jo table mein rows ko uniquely identify kare. Primary Key bhi ek super key hai.
Candidate Key	Minimal Super Key. Ek column bhi remove karo to uniqueness chali jaati hai. Table mein multiple candidate keys ho sakti hain.
Primary Key	Chosen Candidate Key. Jo hum implement karte hain. NOT NULL + UNIQUE.
Composite Key	2 ya zyada columns milke primary key banate hain. Akele koi bhi uniquely identify nahi karta.
Surrogate Key	System-generated artificial key. Real world mein koi meaning nahi. Example: auto-increment id.
Natural Key	Real-world attribute se primary key. Example: Aadhar number, PAN number.

PHASE 4: TRANSACTIONS & CONCURRENCY (~15%)

Goal: Real-world mein multiple users simultaneously database use karte hain. Yeh ensure karna ki sab kuch sahi hoga — yahi transactions ka kaam hai.

4.1 ACID Properties

ACID yeh 4 properties hain jo ensure karte hain ki database transactions reliable hain — especially failures ya concurrent access ke case mein.

A — Atomicity

Transaction ya to puri complete hogi ya bilkul nahi. Beech mein fail ho jaaye to sab rollback ho jaata hai.

```
-- Example: Bank Transfer (Rahul se Priya ko Rs 500)
BEGIN TRANSACTION;
    UPDATE Accounts SET balance = balance - 500 WHERE user = 'Rahul';
    UPDATE Accounts SET balance = balance + 500 WHERE user = 'Priya';
COMMIT;

-- Agar dusra UPDATE fail ho to:
ROLLBACK;  -- Pehla UPDATE bhi undo ho jaata hai
-- Bina atomicity ke: Rahul ka balance kam ho jaata, Priya ko credit nahi
milti!
```

C — Consistency

Transaction se pehle aur baad mein database valid state mein hona chahiye. Saari constraints (Primary Key, Foreign Key, CHECK) satisfy honi chahiye.

- Example: Bank transfer ke baad total balance change nahi hona chahiye (conservation)
- Check constraint violate nahi honi chahiye
- Referential integrity maintain rehni chahiye

I — Isolation

Concurrent transactions ek doosre ko affect nahi karni chahiye. Har transaction aisa behave kare jaise woh akeli chal rahi hai.

- Isolation Levels define karte hain ki kitna isolation chahiye (detail neeche)
- Perfect isolation = Serializability (sabse safe, par slow)

D — Durability

Committed transaction permanent hai. System crash hone par bhi data permanent rehta hai.

- Transaction commit hone ke baad disk par write ho jaata hai
- Yeh Write-Ahead Logging (WAL) ya similar mechanisms se achieve hota hai

4.2 Isolation Levels

SQL standard 4 isolation levels define karta hai — har level kuch problems solve karta hai par kuch performance cost aati hai.

Isolation Level	Dirty Read	Non-Rep Read	Phantom Read
Read Uncommitted	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible
Serializable	Prevented	Prevented	Prevented

4.3 Concurrency Problems

Dirty Read

Transaction T1 ne kuch data write kiya (commit nahi hua). T2 woh data read kar leta hai. Phir T1 rollback ho jaata hai. Ab T2 ke paas invalid data hai.

```
-- T1:  UPDATE balance = 1000 WHERE id = 1;  (not committed)
-- T2:  SELECT balance FROM Accounts WHERE id = 1;  -- reads 1000 (DIRTY!)
-- T1:  ROLLBACK;  -- balance wapas 500 ho jaata hai
-- T2 ne wrong data read kiya tha!
```

Non-Repeatable Read

T1 ek row read karta hai. T2 usi row ko modify aur commit karta hai. Ab T1 dobara same row read karta hai — different value milti hai!

```
-- T1: SELECT price FROM Products WHERE id = 5;  -- reads 100
-- T2: UPDATE Products SET price = 200 WHERE id = 5; COMMIT;
-- T1: SELECT price FROM Products WHERE id = 5;  -- reads 200 (CHANGED!)
-- Same transaction mein alag values — inconsistency!
```

Phantom Read

T1 ek query chalata hai. T2 naye rows insert/delete karta hai. T1 dobara same query chalata hai — result set change ho jaata hai!

```
-- T1: SELECT * FROM Orders WHERE amount > 500;  -- 5 rows milti hain
-- T2: INSERT INTO Orders (cust_id, amount) VALUES (1, 800); COMMIT;
-- T1: SELECT * FROM Orders WHERE amount > 500;  -- ab 6 rows! (PHANTOM!)
```

4.4 Locks

Lock Type	Explanation
Shared Lock (S Lock)	Multiple transactions ek saath ek data read kar sakti hain. Koi bhi write nahi kar sakta jab tak S lock hai. SELECT statement use karta hai.
Exclusive Lock (X Lock)	Sirf ek transaction data access kar sakti hai — na koi read, na koi write. INSERT/UPDATE/DELETE use karta hai.

> 📝 *Note: Deadlock: T1 ne A lock kiya, T2 ka wait kar rahi hai B ke liye. T2 ne B lock kiya, T1 ka wait kar rahi hai A ke liye. Dono wait kar rahe hain — DEADLOCK! DB detect karke ek ko rollback karta hai.*

PHASE 5: INDEXING & PERFORMANCE (~10%)

Goal: Fast systems banana — queries ko optimize karna aur database performance samajhna.

5.1 Index Deep Dive

B-Tree Index

Most common index type. Balanced tree structure maintain karta hai. Har node mein sorted keys hoti hain aur children ko pointers hote hain.

- Range queries support karta hai (>, <, BETWEEN)
- ORDER BY queries mein helpful
- Most databases mein default index type
- Example: SELECT * FROM Students WHERE age > 20 — B-Tree efficiently pages scan karta hai

Hash Index

Hash function use karta hai keys ko hash table mein map karne ke liye.

- Exact match queries mein B-Tree se faster (O(1) lookup)
- Range queries SUPPORT NAHI karta
- Example: SELECT * FROM Users WHERE email = 'rahul@example.com' — perfect use case

5.2 Query Optimization

Execution Plan

Database query parse karke ek execution plan banata hai — yeh decide karta hai ki query kaise execute hogi, kaunse indexes use honge, join order kya hoga.

```
-- Execution plan dekhne ke liye:
EXPLAIN SELECT * FROM Students WHERE city = 'Delhi';
EXPLAIN ANALYZE SELECT * FROM Students WHERE city = 'Delhi';  -- actual time

-- Output mein dekhna hai:
-- type: 'ALL' = full table scan (BAD), 'ref' ya 'index' = index use (GOOD)
-- rows: estimated rows scanned — kam ho to better
-- Extra: 'Using filesort' ya 'Using temporary' = slow operations
```

5.3 Performance Tips

- **SELECT * mat use karo — sirf zaroori columns select karo**

    - SELECT * bahut zyada data network pe transfer karta hai
    - Index cover nahi ho paata (covering index ka faida nahi milta)
- **Proper indexing karo**
    - WHERE, JOIN, ORDER BY mein frequently use hone wale columns par index banao
    - Low cardinality columns (jaise gender) par index mat banao
- **Pagination use karo large data sets ke liye**

```
-- Pagination:
-- Page 1 (rows 1-10):
SELECT * FROM Products ORDER BY product_id LIMIT 10 OFFSET 0;
-- Page 2 (rows 11-20):
SELECT * FROM Products ORDER BY product_id LIMIT 10 OFFSET 10;

-- Keyset Pagination (zyada efficient large datasets ke liye):
SELECT * FROM Products WHERE product_id > 100 ORDER BY product_id LIMIT 10;
```

- **N+1 Problem avoid karo — JOINs use karo multiple queries ki jagah**
    - N+1: Pehle 1 query, phir N rows ke liye N queries — bahut slow
    - Better: 1 query with JOIN jo sab data ek saath le aaye
- **Connection pooling use karo — har request ke liye naya connection mat banao**

PHASE 6: DATABASE DESIGN — REAL WORLD (~20%)

Goal: Real systems ka database design banana. Interviews mein 'Design karo' type questions bahut aate hain.

6.1 Design Approach — Step by Step

- **Step 1 — Requirements samjho: Kya store karna hai? Kaunse operations honge? Read heavy ya write heavy?**
- **Step 2 — Entities identify karo: Real-world objects dhundho (nouns)**
- **Step 3 — Relationships define karo: Entities aapas mein kaise connected hain (1:1, 1:M, M:M)**
- **Step 4 — Attributes add karo: Har entity ki properties aur constraints**
- **Step 5 — Normalize karo: 1NF, 2NF, 3NF check karo. Redundancy hatao.**
- **Step 6 — Optimize karo: Indexes add karo, denormalize karo agar zarurat ho**

6.2 E-Commerce Database Design

Yeh ek complete e-commerce system ka database design hai. Interview mein yeh ek of the most common design question hai.

Tables Overview

Table	Purpose
Users	Customer accounts — registration, login info
Addresses	User ke multiple addresses (billing, shipping)
Categories	Product categories — hierarchical hoti hain
Products	Product catalog — name, price, stock
Product_Images	Product ki multiple images
Cart	User ka current shopping cart
Cart_Items	Cart mein kaunse products
Orders	Placed orders
Order_Items	Order mein kaunse products, kitne, kis price par
Payments	Payment transactions
Reviews	Product reviews aur ratings

Users Table

```
CREATE TABLE Users (
    user_id       INT           PRIMARY KEY AUTO_INCREMENT,
    email         VARCHAR(150)  NOT NULL UNIQUE,
    password_hash VARCHAR(255)  NOT NULL,   -- kabhi plain text nahi
    first_name    VARCHAR(50)   NOT NULL,
    last_name     VARCHAR(50)   NOT NULL,
    phone         VARCHAR(20),
    is_active     BOOLEAN       DEFAULT TRUE,
    created_at    TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    updated_at    TIMESTAMP     DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);

CREATE TABLE Addresses (
    address_id    INT           PRIMARY KEY AUTO_INCREMENT,
    user_id       INT           NOT NULL REFERENCES Users(user_id) ON DELETE
CASCADE,
    address_type  ENUM('home','work','other') DEFAULT 'home',
    street        VARCHAR(200)  NOT NULL,
    city          VARCHAR(100)  NOT NULL,
    state         VARCHAR(100),
    pincode       VARCHAR(10)   NOT NULL,
    country       VARCHAR(50)   DEFAULT 'India',
    is_default    BOOLEAN       DEFAULT FALSE
);
```

Products & Categories

```
CREATE TABLE Categories (
    category_id   INT           PRIMARY KEY AUTO_INCREMENT,
    name          VARCHAR(100)  NOT NULL,
    parent_id     INT           REFERENCES Categories(category_id), -- self-ref
for hierarchy
    description   TEXT,
    is_active     BOOLEAN       DEFAULT TRUE
);
-- Example: Electronics > Mobiles > Smartphones

CREATE TABLE Products (
    product_id    INT           PRIMARY KEY AUTO_INCREMENT,
    category_id   INT           NOT NULL REFERENCES Categories(category_id),
    name          VARCHAR(200)  NOT NULL,
    description   TEXT,
    price         DECIMAL(10,2) NOT NULL CHECK (price >= 0),
    mrp           DECIMAL(10,2),           -- original price
    stock_qty     INT           DEFAULT 0 CHECK (stock_qty >= 0),
    sku           VARCHAR(100)  UNIQUE,    -- Stock Keeping Unit
    brand         VARCHAR(100),
    is_active     BOOLEAN       DEFAULT TRUE,
    created_at    TIMESTAMP     DEFAULT CURRENT_TIMESTAMP
);
```

Cart

```
CREATE TABLE Cart (
    cart_id       INT       PRIMARY KEY AUTO_INCREMENT,
    user_id       INT       NOT NULL UNIQUE REFERENCES Users(user_id), -- 1
user = 1 cart
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);

CREATE TABLE Cart_Items (
    cart_item_id  INT           PRIMARY KEY AUTO_INCREMENT,
    cart_id       INT           NOT NULL REFERENCES Cart(cart_id) ON DELETE
CASCADE,
    product_id    INT           NOT NULL REFERENCES Products(product_id),
    quantity      INT           NOT NULL CHECK (quantity > 0),
    added_at      TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (cart_id, product_id)   -- ek product ek hi baar cart mein
);
```

Orders & Payments — Most Important!

```
CREATE TABLE Orders (
    order_id         INT           PRIMARY KEY AUTO_INCREMENT,
    user_id          INT           NOT NULL REFERENCES Users(user_id),
    shipping_addr_id INT           REFERENCES Addresses(address_id),
    status
ENUM('pending','confirmed','shipped','delivered','cancelled')
                                   DEFAULT 'pending',
    subtotal         DECIMAL(10,2) NOT NULL,
    tax_amount       DECIMAL(10,2) DEFAULT 0,
    shipping_charge  DECIMAL(10,2) DEFAULT 0,
    total_amount     DECIMAL(10,2) NOT NULL,
    notes            TEXT,
    created_at       TIMESTAMP     DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Order_Items (
    order_item_id    INT           PRIMARY KEY AUTO_INCREMENT,
    order_id         INT           NOT NULL REFERENCES Orders(order_id),
    product_id       INT           NOT NULL REFERENCES Products(product_id),
    quantity         INT           NOT NULL CHECK (quantity > 0),
    unit_price       DECIMAL(10,2) NOT NULL,  -- price at time of order
(fixed!)
    total_price      DECIMAL(10,2) NOT NULL   -- quantity * unit_price
);

CREATE TABLE Payments (
    payment_id       INT           PRIMARY KEY AUTO_INCREMENT,
    order_id         INT           NOT NULL REFERENCES Orders(order_id),
    amount           DECIMAL(10,2) NOT NULL,
    method           ENUM('card','upi','netbanking','cod','wallet') NOT NULL,
```

```
    status           ENUM('pending','success','failed','refunded') DEFAULT
'pending',
    transaction_id   VARCHAR(200)  UNIQUE,    -- payment gateway ID
    gateway_response JSON,                    -- full gateway response store
karo
    created_at       TIMESTAMP     DEFAULT CURRENT_TIMESTAMP
);
```

Common Queries on E-Commerce DB

```
-- 1. Kisi user ke sare orders:
SELECT o.order_id, o.status, o.total_amount, o.created_at
FROM Orders o
WHERE o.user_id = 42
ORDER BY o.created_at DESC;

-- 2. Order ki details — products ke saath:
SELECT p.name, oi.quantity, oi.unit_price, oi.total_price
FROM Order_Items oi
JOIN Products p ON oi.product_id = p.product_id
WHERE oi.order_id = 101;

-- 3. Total revenue is month:
SELECT SUM(total_amount) AS revenue
FROM Orders
WHERE status = 'delivered'
  AND MONTH(created_at) = MONTH(NOW())
  AND YEAR(created_at) = YEAR(NOW());

-- 4. Top 5 selling products:
SELECT p.name, SUM(oi.quantity) AS total_sold
FROM Order_Items oi
JOIN Products p ON oi.product_id = p.product_id
JOIN Orders o ON oi.order_id = o.order_id
WHERE o.status = 'delivered'
GROUP BY p.product_id, p.name
ORDER BY total_sold DESC
LIMIT 5;

-- 5. Cart abandon karne wale users (cart mein items hain, recent order nahi):
SELECT u.email, COUNT(ci.cart_item_id) AS items_in_cart
FROM Users u
JOIN Cart c ON u.user_id = c.user_id
JOIN Cart_Items ci ON c.cart_id = ci.cart_id
WHERE u.user_id NOT IN (
    SELECT DISTINCT user_id FROM Orders
    WHERE created_at > DATE_SUB(NOW(), INTERVAL 7 DAY)
)
GROUP BY u.user_id, u.email;
```

Recommended Indexes for E-Commerce

```sql
-- Users par:
CREATE INDEX idx_users_email ON Users(email);

-- Products par:
CREATE INDEX idx_products_category ON Products(category_id);
CREATE INDEX idx_products_name ON Products(name);
CREATE INDEX idx_products_price ON Products(price);

-- Orders par:
CREATE INDEX idx_orders_user ON Orders(user_id);
CREATE INDEX idx_orders_status ON Orders(status);
CREATE INDEX idx_orders_date ON Orders(created_at);

-- Order_Items par:
CREATE INDEX idx_orderitems_order ON Order_Items(order_id);
CREATE INDEX idx_orderitems_product ON Order_Items(product_id);
```

QUICK REFERENCE — Interview Prep

Most Common Interview Questions

Question	Short Answer
DBMS vs RDBMS?	RDBMS mein data tables mein, relationships hain, SQL use hoti hai
Primary vs Foreign Key?	PK: row uniquely identify karta hai, NOT NULL+UNIQUE. FK: doosri table ki PK ko refer karta hai
ACID kya hai?	Atomicity, Consistency, Isolation, Durability — transaction reliability ensure karte hain
Normalization kyu?	Data redundancy aur anomalies (insert/update/delete) hatane ke liye
JOIN types?	INNER: dono mein match. LEFT: left sab + right match. RIGHT: right sab + left match. SELF: khud se khud
Index kab use?	WHERE, JOIN, ORDER BY mein frequently use hone wale columns par
Clustered vs Non-clustered?	Clustered: data physically sort hota hai (1 per table). Non-clustered: separate structure, multiple allowed
Deadlock kya hai?	Do transactions ek doosre ka wait karti hain — dono stuck ho jaati hain
Dirty Read?	Uncommitted data padhna — Transaction rollback ho to invalid data
Phantom Read?	Same query alag results deti hai — beech mein naye rows insert/delete hue

SQL Execution Order

SQL query likhte waqt yeh order mein execute hoti hai — ALWAYS yaad rakho:

```
1. FROM       -- Table(s) choose karo
2. JOIN       -- Tables join karo
3. WHERE      -- Rows filter karo
4. GROUP BY   -- Groups banao
5. HAVING     -- Groups filter karo
6. SELECT     -- Columns choose karo
7. DISTINCT   -- Duplicates hatao
```

```
8. ORDER BY    -- Sort karo
9. LIMIT       -- Result limit karo

-- ISLIYE: WHERE mein aliases use nahi kar sakte (SELECT abhi hua nahi)
-- ISLIYE: ORDER BY mein SELECT aliases use kar sakte hain
```

Normalization Quick Summary

Normal Form	Rule
1NF	Har cell atomic ho. No repeating groups.
2NF	1NF + No partial dependency (composite PK hone par)
3NF	2NF + No transitive dependency (non-key -> non-key)
BCNF	3NF + Har determinant super key hona chahiye

All the best! Keep practicing SQL daily.